

The following representative outputs were deliberately selected from a set of ten open-ended problems involving applications of first-order Ordinary Differential Equations. They are intended to illustrate both the strengths and limitations of Large Language Models in mathematical modeling, Python code generation, solution accuracy, and the generation of analogous problems. The observed performance vary across different Large Language Models.

Question ID:1.QUESTION TYPE-Newton's Law of Cooling:
A body at an unknown temperature is placed in a room which is held at a constant temperature of 30°F. If after 10 minutes the temperature of the body is 0°F and after 20 minutes the temperature of the body is 15°F, find the unknown initial temperature?

Ground Truth Solution:
Let T denote the temperature of the body. Then Newton's law of cooling can be formulated as $dT/dt = -k(T - T_m) \Rightarrow dT/dt + kT = kT_m$. Let $T_m = 30$, $dT/dt + kT = 30k$. Solving, we obtain $T = ce^{(-kt)} + 30$. At $t = 10$, we are given that $T = 0$. Hence, $0 = ce^{(-10k)} + 30 \Rightarrow ce^{(-10k)} = -30$. At $t = 20$, we are given that $T = 15$. Hence, $15 = ce^{(-20k)} + 30 \Rightarrow ce^{(-20k)} = -15$. Solving for k and c , we find $k = (1/10)\ln 2 = 0.069$ and $c = -30e^{(10k)} = -30(2) = -60$. Substituting these values, we have $T = -60e^{(-0.069t)} + 30$. Since we require T at $t = 0$, it follows that $T_0 = -60e^0 + 30 = -60 + 30 = -30^\circ\text{F}$.

Ground Truth Answer: -30

Response: **Deepseek-r1**
Solution 1: Modeling the problem using an ODE
The temperature of the body follows Newton's Law of Cooling: $dT/dt = k(30 - T)$, where $T(t)$ is the temperature at time t and k is the cooling constant. The general solution is $T(t) = 30 + (T_0 - 30)e^{-kt}$. Using the data points $T(10) = 0$ and $T(20) = 15$, we solve two equations to find T_0 and k . By dividing the equations, we find $k = \ln(2)/10 \approx 0.0693$. Substituting back, $T_0 = -30^\circ\text{F}$. 

Solution 02: Convert the ODE model into Python code

```
python\nfrom scipy.integrate import solve_ivp\nfrom scipy.optimize import fsolve\n\ndef model(t, T, k):\n    return k * (30 - T)\n\nequations(params):\n    T0, k = params\n    sol = solve_ivp(model, (0, 20), [T0], args=(k,), t_eval=[10, 20], rtol=1e-6, atol=1e-6)\n    return (sol.y[0][0], sol.y[0][1] - 15)\n\nT0, k = fsolve(equations, (-30.0, 0.0693))\n\nprint("<result>", round(T0, 5), "</result>")
```



Answer: Error 

Solution 03: Generate two (2) new analogous problems
1. **QUESTION TYPE-Newton's Law of Cooling**: A cup of coffee cools in a room at 25°C. After 8 minutes, its temperature drops to 60°C from an initial 85°C. What was its temperature 4 minutes after being placed in the room? 

2. **QUESTION TYPE-Newton's Law of Heating**: A metal rod at 10°C is heated in a furnace at 200°C. After 3 minutes, it reaches 50°C. How long will it take to reach 150°C? 

Figure 1: llm generated ODE model,python code and two analogous problems- newtons law of cooling problem

Question ID:02.QUESTION TYPE-Mixing Problem:
A tank contains 1000 L of salt water with 15 kg of dissolved salt. Pure water enters the tank at a rate of 10 L/min. The solution is kept well-mixed and drains from the tank at the same rate. Calculate how much salt remains in the tank after 20 minutes.

Ground Truth Solution:
Let $S(t)$ be the amount of salt (in kilograms) in the tank at time t (in minutes). Since the inflow and outflow rates are equal, the volume in the tank remains constant at 1000 liters. We are given that: Inflow of pure water; no salt is added; Outflow: the concentration of salt at time t is $S(t)/1000$ kg/L; Outflow rate of salt: $10 \cdot S(t)/1000 = 1/100 S(t)$ kg/min. Thus, the rate of change of salt in the tank is: $dS/dt = -1/100 S(t) \rightarrow dS/S = -1/100 dt$ Integrate both sides: $\int 1/S dS = \int -1/100 dt \rightarrow \ln |S| = -t/100 + C \rightarrow S(t) = Ce^{-(t/100)}$ Applying the initial condition $S(0) = 15$, we have: $S(0) = C \cdot e^0 = C \rightarrow C = 15$ Hence, $S(t) = 15 e^{-(t/100)}$ Calculating the amount of salt at $t = 20$, we have $S(20) = 15 e^{-(20/100)} = 15 e^{-(0.2)}$ Using the approximation $e^{-(0.2)} \approx 0.8187$: $S(20) \approx 15 \cdot 0.8187 = 12.28$ kg Therefore, after 20 minutes, we have approximately 12,3 kg"

Ground Truth Answer: 12.3

Response: **Gpt-5.2**

```
## Task 01: ODE model (step-by-step, plain text math)
1. Define the state variable
Let  $y(t)$  = amount of salt in the tank (kg) at time  $t$  (minutes).
2. Determine the volume behavior
Inflow rate = 10 L/min and outflow rate = 10 L/min, so the volume stays constant:
 $V = 1000$  L.
3. Write the salt inflow rate
The inflow is pure water, so inflow salt concentration = 0 kg/L.
Salt inflow rate =  $(0 \text{ kg/L}) \cdot (10 \text{ L/min}) = 0$  kg/min.
4. Write the salt outflow rate
The tank is well-mixed, so the salt concentration in the tank is:
 $\text{concentration} = y(t) / 1000$  kg/L.
Salt outflow rate =  $(y(t) / 1000 \text{ kg/L}) \cdot (10 \text{ L/min}) = y(t) / 100$  kg/min.
5. Form the differential equation (rate of change = inflow - outflow)
 $dy/dt = 0 - y/100$ 
 $dy/dt = -y/100$ 
6. Initial condition
 $y(0) = 15$  (kg)
7. Symbolic solution form (do not numerically evaluate)
 $y(t) = 15 \cdot \exp(-t/100)$ 
8. Quantity requested
Salt after 20 minutes is  $y(20) = 15 \cdot \exp(-20/100)$ 
```

Task 02: Executable Python code (solve_ivp; outputs final value rounded to 5 decimals)
python
from scipy.integrate import solve_ivp
def dydt(t, y):
 return [-y[0] / 100.0]
sol = solve_ivp(dydt, [0.0, 20.0], [15.0], t_eval=[20.0])
result = round(float(sol.y[0][-1]), 5)
print(result)

Answer: 12.28096

Task 03: Two new analogous problems
1) Mixing Problem A
A tank contains 800 L of salt water with 24 kg of dissolved salt. Pure water flows into the tank at 8 L/min. The well-mixed solution flows out at the same rate. How much salt remains in the tank after 30 minutes?
2) Mixing Problem B
A tank contains 1200 L of water with 10 kg of dissolved salt. Salt water with concentration 0.05 kg/L flows in at 12 L/min. The well-mixed solution flows out at the same rate. How much salt is in the tank after 25 minutes?

Figure 2: llm generated ODE model,python code and two analogous problems-mixing problem

Question ID:4.
 QUESTION TYPE-Exponential Decay: A certain village in Angola had a population of 10,000 people. Due to unfavorable living conditions, the population decreases at a rate proportional to the current population. After 3 years, the population decreased to 8,000. Determine the time it will take for the population to reduce to 5,000?

Ground Truth Solution:
 "Let $P(t)$ be the population at time t . The rate of population decrease is proportional to the population: $dP/dt = -kP$ where k is the decay constant. Solving this ODE, we have $P(t) = Ae^{(-kt)}$. Applying initial condition, $P(0) = 10,000 \rightarrow P(t) = 10,000e^{(-kt)}$. After 3 years, the population is 8,000. Substituting $P(3) = 8,000$ into the equation and calculating the value of k , we have $8,000/10,000 = e^{(-3k)} \rightarrow \ln(0.8) = -3k \rightarrow k = -\ln(0.8)/3 \approx 0.0749$. Hence $P(t) = 10,000e^{(-0.0749t)}$. Now setting $P(t) = 5,000$ and solving for t , we have $5,000 = 10,000e^{(-0.0749t)} \rightarrow 5,000/10,000 = e^{(-0.0749t)} \rightarrow t = -\ln(0.5)/0.0749 \approx 9.26$ years. Therefore it will take approximately 9.26 years for the population to reduce to 5,000 people.

Ground Truth Answer: 9.26

Response: Claude-haiku-4.5

Solution to Exponential Decay Problem\n\n## Task 01: Modeling the problem using an ODE\n\n**Step 1: Define the variable**\nLet $P(t)$ be the population at time t (in years).\n\n**Step 2: Formulate the differential equation**\nSince the population decreases at a rate proportional to the current population:\n
$$\frac{dP}{dt} = -kP$$
where k is the decay constant ($k > 0$).\n\n**Step 3: Solve the differential equation**\nThis is a separable ODE. The general solution is:\n
$$P(t) = A e^{(-kt)}$$
\n\n**Step 4: Apply initial condition**\nAt $t = 0$, $P(0) = 10,000$, so:\n
$$10,000 = A e^{(-k \cdot 0)} = A$$
\n\n**Step 5: Find the decay constant k **\nWe are given that $P(3) = 8,000$. Substituting:\n
$$8,000 = 10,000 e^{(-k \cdot 3)} \Rightarrow 0.8 = e^{(-3k)} \Rightarrow \ln(0.8) = -3k \Rightarrow k = -\ln(0.8)/3 \approx 0.0744$$
\n\n**Step 6: Determine the time when population reaches 5,000**\nSet $P(t) = 5,000$ and solve for t :\n
$$5,000 = 10,000 e^{(-0.0744t)} \Rightarrow 0.5 = e^{(-0.0744t)} \Rightarrow \ln(0.5) = -0.0744t \Rightarrow t = -\ln(0.5)/0.0744 \approx 9.31$$
 years\n\nTherefore, it will take approximately 9.31 years for the population to reduce to 5,000.\n\n---\n\n## Task 02: Python Code Solution\n\n```\npython\nfrom scipy.integrate import solve_ivp\nimport numpy as np\nfrom scipy.optimize import fsolve\n\ndef population_decay(t, y, k):\n return [-k * y[0]]\n\ndef find_k(k_guess):\n sol = solve_ivp(population_decay, [0, 3], [10000], args=(k_guess,), t_eval=[3])\n return sol.y[0][0] - 8000\n\nk = fsolve(find_k, 0.1)[0]\n\ndef population_decay_final(t, y):\n return [-k * y[0]]\n\nsol = solve_ivp(population_decay_final, [0, 100], [10000], dense_output=True)\nt_eval = np.linspace(0, 100, 10000)\nP_eval = sol.sol(t_eval)[0]\ntime_to_5000 = t_eval[np.argmin(np.abs(P_eval - 5000))]\nresult = round(float(time_to_5000), 5)\nprint("<result>", result, "</result>")\n```\n\n---\n\nAnswer: 11.997 ✖

Task 03: Two New Analogous Problems\n\n## **Problem 1: Radioactive Decay**\nA radioactive substance has an initial mass of 500 grams. Due to radioactive decay, the mass decreases at a rate proportional to the current mass. After 10 years, the mass has decreased to 400 grams. How long will it take for the mass to reduce to 250 grams?\n\n---\n

Problem 2: Drug Concentration in Bloodstream\nA patient is administered a drug intravenously, and the initial concentration in the bloodstream is 100 mg/L. The drug is eliminated from the body at a rate proportional to its current concentration. After 2 hours, the concentration decreases to 75 mg/L. Determine how many hours it will take for the drug concentration to reduce to 50 mg/L? ✓

Figure 3: llm generated ODE model,python code and two analogous problems- exponential decay problem